

Task sheet 4 – Artificial Neural Networks

Deadline : 03.12.2025

Objectives:

- ▶ implementation of ANN with hidden layers
- ▶ influence of bias neurons and other parameters
- ▶ avoid unwanted behaviors & push towards desired behaviors
- ▶ influence of initial robot positioning

1 Evolve an ANN that solves the XOR problem

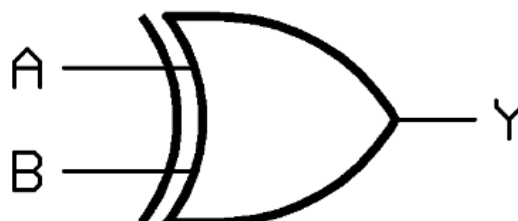
Solving the XOR problem with an ANN requires a hidden layer. Implement an ANN with 2 input neurons, 2 hidden neurons, and 1 output neuron. As activation function take the following sigmoid function

$$\phi(x) = \frac{2}{(1 + \exp(-2x))} - 1,$$

which allows negative outputs. Implement an evolutionary algorithm of your choice and the ANN as described above. The fitness function tests for all 4 possible inputs $I = \{(0, 0), (1, 0), (0, 1), (1, 1)\}$ and sums over the deviations

$$F(ANN) = \frac{1}{4} \sum_{(a,b) \in I} 1 - |(a \text{ XOR } b) - ANN(a,b)|.$$

whereas $ANN(a, b)$ represents the output of the evaluated ANN; the output is a floating point number. Use a relatively high population size of 1000 or more and do up to 2000 generations if necessary. Stop the evolution when a fitness $F > 0.99$ has been reached. Plot the output (activation of the output neuron) of a successful ANN for the continuous input space $[0, 1]^2$.



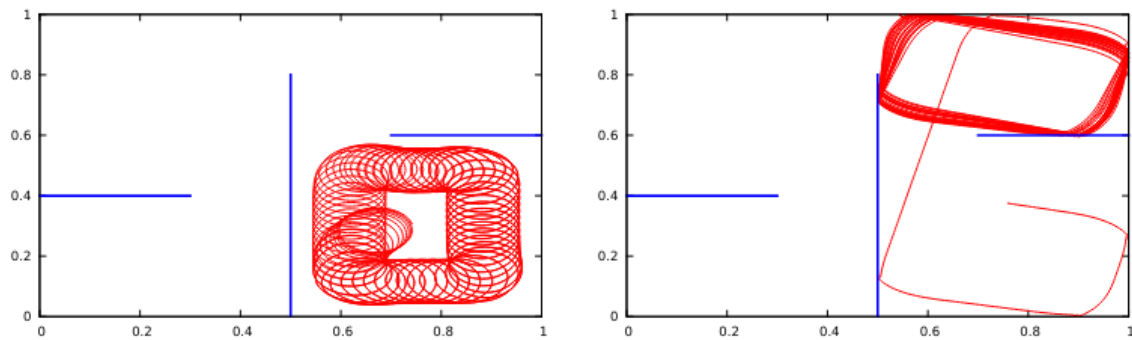
For task 1 please turn in:

- ▶ a single .zip file named:
`evoRobo_sheet4_YOURLASTNAME1_YOURLASTNAME2.zip`
- ▶ a readme file with full group member names and completed tasks
- ▶ accepted plot formats: png, jpg
- ▶ all your code (C/C++ or Python)
- ▶ plot of best and average fitness over generations
- ▶ plot of output neuron activations over $[0, 1]^2$
- ▶ optional but preferred: a video explaining your code

2 Solving the exploration task with an ANN

Finally we get back to our robot simulator. The task is the same as in tutorial 2, task 2: exploration. For now place the robot at a fixed position with fixed heading initially in each evaluation as before. This time the robot controller is implemented as an ANN with hidden layer. Implement an ANN with 3 input neurons, 2 neurons in the hidden layer, and 2 output neurons (one for each wheel). In addition, implement an evolutionary algorithm of your choice or reuse your code. Try to make good decisions on the length of evaluations, the population size, etc. The fitness is again based on visited grid cells. This time, use a grid with high resolution of at least 100×100 .

- a) Do a number of independent evolutionary runs, log best and average fitness, and plot them.
- b) Experiment a little with adding/deleting bias neurons that allow to shift the input from the sensors. Does that change the resulting behaviors qualitatively? What about the initialization of weights or the maximal angular velocity of the robot?
- c) Maybe most of your evolved robots go in spirals/circles. Try to add a component to your fitness function that punishes too intensive turning behavior. In turn, if your robots tend to go in straight lines, try to make them go in spirals. If your robots tend to move close to walls, try to make them keeping a bigger distance. Try to tweak your fitness function (you can also experiment with different weighting of the components in the fitness function) until you see the desired change in the robots' behaviors.
- d) Finally, place the robot at a random position and with random heading for each evaluation. This changes the whole approach qualitatively as we now have a non-deterministic fitness function (i.e., evaluating the very same ANN twice might give two different fitness values). Do a number of independent evolutionary runs, log best and average fitness, and plot them. What has changed in comparison to the deterministic approach? You can also test your best evolved controller started from different positions after evolution has been done.



(placeholder for robot trajectory images)

For task 2 please turn in:

- ▶ all your code (C/C++ or Python)
- ▶ a plot of a typical robot trajectory for a successful deterministic run
- ▶ a plot of best and average fitness over generations (deterministic)
- ▶ robot trajectory plots showing qualitative changes due to bias neurons
- ▶ description of modifications to the fitness function and resulting trajectories
- ▶ plot of best and average fitness for a successful non-deterministic run
- ▶ description of observations from random initializations
- ▶ a trajectory plot starting in a completely different region of the arena